Per eseguire un progetto di sviluppo in ambiente containerizzato utilizzo uno script che in base al tipo di progetto (Python, Rust, Java etc) identifica l' immagine da utilizzare ed in base al nome della directory del progetto identifica il nome del container, questo poi esegue degli script, tipicamente build.sh che può utilizzare un Dockerfile ed un entrypoint.sh come templates, e run.sh che monta la cartella di progetto nel container ed eventuali volumi (venv, node_modules), quindi avvia il container.

Il comando:
Ddev --target=Python/foo
Usa:
image_name=dev.python
container_name=dev.python.foo

Crea l'immagine (usa argomenti nel Dockerfile, entrypoint.sh, .bashrc) se non esiste ed avvia il container con i mount previsti.

Attualmente il container viene sempre avviato con -rm -it
Quali sono i pro e contro nel rimuovere e ricreare il container oppure stopparlo e riavviarlo?

Per un ambiente di sviluppo, **ricreare il container** (`--rm`) è generalmente la scelta migliore se tutti gli artefatti importanti sono su bind mount o volumi; **stop/start** è utile quando vuoi conservare stato interno del container, sessioni di shell, processi in background o file fuori dai mount. Docker rimuove il filesystem scrivibile e i metadati del container con `--rm`/`docker rm`, mentre `docker stop` + `docker start` preserva entrambi.[1] [2]

Ricreare il container

È il modello *ephemeral*: ogni `Ddev --target=...` crea un container pulito dalla stessa immagine, con gli stessi mount e variabili d'ambiente dichiarate nello script.

Pro

- **Riproducibilità**: ogni sessione parte da una configurazione nota; niente pacchetti installati “a mano”, variabili modificate, file di cache o processi dimenticati che alterano silenziosamente l’ambiente.
- **Coerenza con l’immagine**: se aggiorni `dev.python`, `dev.rust`, ecc., la ricreazione garantisce che il nuovo container usi davvero la nuova immagine. Un container fermo non acquisisce automaticamente aggiornamenti dell’immagine.
- **Niente accumulo di container**: eviti il classico `docker ps -a` pieno di container exited e il rischio di conflitti sul nome `dev.python.foo`.
- **Debug più pulito**: se un problema scompare ricreando il container, hai subito un indizio che dipendeva da stato locale, non dal codice o dalla configurazione dichiarata.
- **Ideale per la tua architettura**: `venv`, `node_modules`, cache di build e directory progetto dovrebbero già essere mount; in tal caso il filesystem interno del container contiene poco di prezioso.

Contro

- **Perdi il writable layer**: file creati dentro il container ma non in un volume/bind mount spariscono alla rimozione. [2] [1]
- **Perdi lo stato di runtime**: shell aperte, variabili esportate, daemon in background, socket, PID, sessioni tmux interne, cron locali, ecc.
- **Perdi i log del container**: con `--rm` non puoi più fare `docker logs` dopo l’uscita; per diagnosticare crash devi scrivere i log su file montato o usare un volume.
- **Può essere leggermente più lento**: creare il container richiede inizializzazione di namespace, rete e mount; per progetti molto grandi o con molti mount la differenza può essere percepibile, anche se di solito è modesta. [3]
- **Richiede disciplina sui mount**: se qualcosa di utile finisce in `/root`, `/tmp`, `/opt`, o nella home interna del container, verrà perso.

Stop e riavvio

`docker stop` termina il processo principale con un graceful shutdown; il container resta fermo ma conserva filesystem, configurazione, rete dichiarata e log. `docker start` lo riporta in vita senza ricreare mount o variabili. [4] [5] [1]

Pro

- **Persistenza dello stato interno**: utile per tool che scrivono nella home del container, cache non montate, database locali di sviluppo, daemon, shell interattive con cronologia, ecc.
- **Avvio potenzialmente più rapido**: non ricrea container, rete e mount; riesegue il comando/entrypoint già configurato.
- **Log conservati**: puoi ispezionare `docker logs dev.python.foo` anche dopo l’arresto.
- **Utile per servizi long-running**: ad esempio un container con database, watcher, dev server o agent che vuoi fermare e riprendere.

Contro

- **Deriva dell'ambiente**: il container può accumulare pacchetti, file, configurazioni e processi che non sono rappresentati da Dockerfile, entrypoint o script. Dopo settimane, "funziona nel mio container" può non essere riproducibile.
- **Configurazione congelata**: `docker start` riusa le opzioni del `docker run` originale. Se cambi immagine, mount, porte, variabili o rete nello script, devi comunque rimuovere e ricreare il container. [5]
- **Rischio di stato corrotto**: processi interrotti male, lock, socket, cache incomplete o database in stato incoerente possono persistere.
- **Gestione del ciclo di vita più complessa**: il tuo script deve distinguere tra "container esiste e gira", "esiste ma fermo", "esiste ma con configurazione obsoleta", "non esiste".
- **Consumo di metadati/disco**: i container fermi conservano il proprio writable layer e log finché non vengono rimossi. [6] [1]

Impatto sui tuoi mount

Dato	`--rm` + ricreazione	`stop` + `start`
Directory progetto in bind mount	Conservata	Conservata
`venv`, `node_modules`, cache su volume/bind mount	Conservati	Conservati
Pacchetti installati dentro il container	Persi	Conservati
File in `/root`, `/tmp`, `/home/...` non montati	Persi	Conservati
Variabili esportate in una shell interattiva	Perse	Perse dopo lo stop; conservate solo se parte della config del container
Processi in background	Terminati	Riavviati solo se l'entrypoint li lancia di nuovo
Log Docker del container	Persi dopo l'uscita	Conservati

I volumi sono indipendenti dal ciclo di vita del container: sopravvivono sia alla ricreazione sia alla rimozione, purché siano rimontati nel nuovo container. [2]

Consiglio pratico per `Ddev`

Ti suggerirei un comportamento ibrido, con **ricreazione come default**:

1. **Default: ricrea.**
   Per `build.sh`, test, lint, esecuzioni una-tantum e sessioni di sviluppo normali, usa `--rm`. È il modello più pulito e riproducibile.
2. **Usa stop/start solo per ambienti "residenti".**
   Ad esempio un container con database, dev server, watcher o strumenti con cache pesante. Puoi esporlo con una modalità esplicita:
   `Ddev --target=Python/foo --keep` oppure `Ddev --target=Python/foo --service`.

3. **Non affidarti mai al writable layer per dati importanti.**
   Monta esplicitamente:
   - progetto: bind mount
   - `venv`, `node_modules`, `target/`, `.gradle`, `.m2`, cache: volumi nominati o bind mount
   - log: file o directory montata
   - configurazioni utente: bind mount di dotfiles selezionati
4. **Ricrea automaticamente se la configurazione è cambiata.**
   Calcola un hash di immagine, Dockerfile/args, entrypoint, `.bashrc`, mount, env e porta. Se cambia, rimuovi il container esistente e ricrealo. Questo evita il principale svantaggio dello stop/start.
5. **Considera `docker compose` o un piccolo wrapper dichiarativo.**
   Per progetti con servizi, mount e volumi, Compose rende più esplicita la distinzione tra immagine, configurazione, volumi persistenti e container effimero.

In sintesi: per il tuo caso, con `venv` e `node_modules` già gestiti come volumi, `--rm` è **probabilmente la scelta corretta**. Mantieni stop/start come opt-in per container di sviluppo long-running, non come comportamento predefinito. [1] [2]

⁂

1. https://docs.docker.com/reference/cli/docker/container/run/
2. https://docs.docker.com/get-started/docker-concepts/running-containers/persisting-container-data/
3. https://github.com/moby/moby/issues/38077
4. https://docs.docker.com/reference/cli/docker/container/
5. https://techearl.com/docker-container-lifecycle
6. https://awjunaid.com/docker/how-to-create-start-stop-and-remove-docker-containers-essential-container-lifecycle-commands/
7. https://stackoverflow.com/questions/25775266/how-to-keep-docker-container-running-after-starting-services
8. https://www.imcseian.com/2026/09/container-lifecycle-run-stop-remove.html
9. https://devops.trainwithsky.com/blog/docker/basic-docker-commands
10. https://dockerhol.com/blog/docker-container-lifecycle-create-run-pause-stop-clean
11. https://dockercontainer.it/docker-containers/container-lifecycle-e-persistenza-dei-dati
12. https://www.reddit.com/r/docker/comments/z4i7mz/is_it_possible_to_just_start_a_container_and_keep/
13. https://www.dash0.com/faq/how-to-preserve-data-when-a-docker-container-exits
14. https://labs.iximiuz.com/challenges/docker-101-container-stop-and-restart
15. https://last9.io/blog/docker-stop-vs-kill/